

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI **Semester: 5 Academic Year: 2026-27**

Division: E **Batch: 1** **Practical No: 1**

Roll no: 34 **PRN No: 12412924**

Name of Student: Rushikesh Subhash Rathod

Subject Name : Deep Learning

Problem Statement

Build and train a Convolutional Neural Network (CNN) using TensorFlow/Keras on the MNIST handwritten digit dataset. Perform data preprocessing, normalization, model training, evaluation, visualization of training performance, and prediction of handwritten digits.

Algorithm

1. Import the required libraries such as TensorFlow, NumPy, and Matplotlib.
2. Load the MNIST handwritten digit dataset.
3. Normalize the pixel values by dividing them by 255.
4. Reshape the dataset to include a channel dimension for CNN input.
5. Display the shapes of the training and testing datasets.
6. Create a Convolutional Neural Network (CNN) using Keras Sequential API.
7. Add Convolution, MaxPooling, Flatten, Dense, and Dropout layers.
8. Compile the model using the Adam optimizer and Sparse Categorical Crossentropy loss function.
9. Train the model for 5 epochs using the training dataset.
10. Evaluate the trained model on the testing dataset.

11. Plot the training and validation accuracy and loss graphs.
12. Predict the classes of sample test images and display the predicted and actual labels.

Code

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

# Load MNIST Dataset
mnist = tf.keras.datasets.mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape data for CNN
x_train = x_train[..., tf.newaxis]
x_test = x_test[..., tf.newaxis]

print("x_train shape:", x_train.shape)
print("y_train shape:", y_train.shape)
print("x_test shape:", x_test.shape)
print("y_test shape:", y_test.shape)

# Build CNN Model
```

```
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu',
input_shape=(28,28,1)),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

```
# Display Model Summary
```

```
model.summary()
```

```
# Compile Model
```

```
model.compile(
    optimizer='adam',
```

```
loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=['accuracy']
)
```

```
# Train Model
```

```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
```

```
validation_data=(x_test, y_test)
)

# Evaluate Model
loss, accuracy = model.evaluate(x_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

# Plot Accuracy and Loss
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])

plt.subplot(1,2,2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
```

```
plt.tight_layout()
plt.show()

# Prediction
predictions = model.predict(x_test[:5])
predicted_classes = np.argmax(predictions, axis=1)

print("Predicted Classes:", predicted_classes)
print("Actual Labels:", y_test[:5])

# Display Images
plt.figure(figsize=(10,4))

for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title(f"P:{predicted_classes[i]}\nT:{y_test[i]}")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Output

```
import tensorflow as tf
import matplotlib.pyplot as plt
```

1. Load and Preprocess the MNIST Dataset

```
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape data to add a channel dimension (for CNN input)
x_train = x_train[..., tf.newaxis]
x_test = x_test[..., tf.newaxis]

# Display some information about the dataset
print(f"x_train shape: {x_train.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"x_test shape: {x_test.shape}")
print(f"y_test shape: {y_test.shape}")

... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
```

```
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.summary()
```

```
... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0

3. Compile and Train the Model

```
▶ model.compile(optimizer='adam',
                loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
                metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))

... Epoch 1/5
1875/1875 ————— 65s 34ms/step - accuracy: 0.9368 - loss: 0.2049 - val_accuracy: 0.9815 - val_loss: 0.0525
Epoch 2/5
1875/1875 ————— 78s 32ms/step - accuracy: 0.9768 - loss: 0.0770 - val_accuracy: 0.9889 - val_loss: 0.0371
Epoch 3/5
1875/1875 ————— 59s 31ms/step - accuracy: 0.9840 - loss: 0.0549 - val_accuracy: 0.9909 - val_loss: 0.0257
Epoch 4/5
1875/1875 ————— 83s 32ms/step - accuracy: 0.9866 - loss: 0.0450 - val_accuracy: 0.9913 - val_loss: 0.0267
Epoch 5/5
1875/1875 ————— 81s 32ms/step - accuracy: 0.9891 - loss: 0.0369 - val_accuracy: 0.9913 - val_loss: 0.0264
```

4. Evaluate the Model

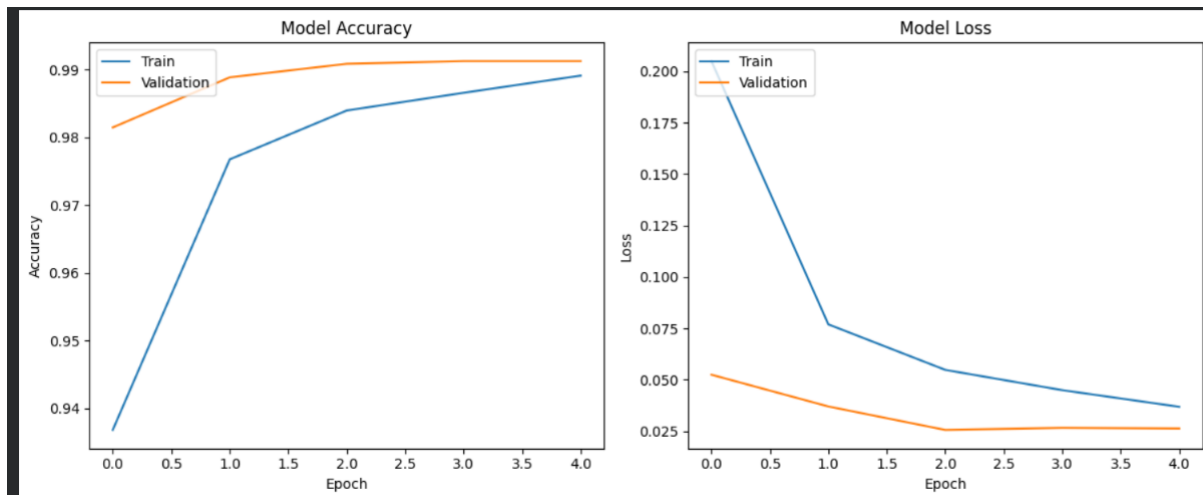
```
loss, accuracy = model.evaluate(x_test, y_test, verbose=2)
print(f"Test loss: {loss:.4f}")
print(f"Test accuracy: {accuracy:.4f}")
```

```
313/313 - 3s - 9ms/step - accuracy: 0.9913 - loss: 0.0264
Test loss: 0.0264
Test accuracy: 0.9913
```

```
▶ # Plot training history (accuracy and loss)
plt.figure(figsize=(12, 5))

# Plot training & validation accuracy values
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

# Plot training & validation loss values
plt.subplot(1, 2, 2)
```



```

predictions = model.predict(x_test[:5])
predicted_classes = np.argmax(predictions, axis=1)

print("Predicted classes for the first 5 test images:")
print(predicted_classes)
print("True labels for the first 5 test images:")
print(y_test[:5])

# Display the images and their predictions
plt.figure(figsize=(10, 4))
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title(f"Pred: {predicted_classes[i]}\nTrue: {y_test[i]}")
    plt.axis('off')
plt.tight_layout()
plt.show()

```

1/1 — 0s 95ms/step
Predicted classes for the first 5 test images:
[7 2 1 0 4]
True labels for the first 5 test images:
[7 2 1 0 4]

Pred: 7 True: 7	Pred: 2 True: 2	Pred: 1 True: 1	Pred: 0 True: 0	Pred: 4 True: 4

Result

The Convolutional Neural Network (CNN) was successfully implemented using TensorFlow and Keras. The MNIST handwritten digit dataset was preprocessed, normalized, and reshaped for CNN input. The model was trained for five epochs, achieving high classification accuracy on the test dataset. The training and validation accuracy and loss were visualized, and the trained model correctly predicted the handwritten digits from the test dataset.

